Notes on the Self-Compiling C Compiler (SCCC)

by
Ivan Gaydardzhiev

ABSTRACT

SCCC is a self-compiling C compiler and x86-64 toolchain. Its C parser emits machine code as it reads the source; the assembler uses the same instruction encoder. The toolchain writes relocatable ELF objects and executables, reads archives, interprets GNU ld scripts, and produces flat and bootable images. It also links ELF32 boot code and shared ELF objects. SCCC compiles itself to identical second- and third-stage binaries. It builds xv6-x86_64 and Linux 5.4.302, together with musl 1.2.5 and BusyBox 1.36.1; the Linux system boots to an interactive BusyBox shell. These notes describe the compiler, assembler, linking and runtime machinery, kernel build interfaces, and operating-system builds.

1. Introduction

SCCC accepts C files, assembly source, ELF relocatable objects, and archives. It generates x86-64 machine instructions and writes ELF executables, relocatable objects, flat binaries, BIOS disk images, and Multiboot images. Its assembler and linker are part of the same program. A GNU-style linker script can give sections their addresses in memory and in the output file.

A hosted compilation reads the input files, the embedded headers, and the embedded C library. With `-ffreestanding`, it omits the hosted startup and library. With `-c`, it emits ELF objects for later linking. The compiler also reads `.s` source as assembly and preprocesses `.S` before assembling it. References between files are resolved during the final link.

The C front end uses recursive descent for expressions, declarations, and statements. It constructs types while parsing and emits instructions while reading expressions. A normal expression returns a value or an address in `RAX`; the `lv` state distinguishes the two. The C emitter and the assembler share the machine-code encoder. They feed section buffers and relocation records to the output stage.

The compiler is itself written in C99. GCC builds the first executable, which compiles `sccc.c` into a second compiler. The second produces a third. The last two executables are compared byte for byte. SCCC builds xv6-x86_64 and Linux 5.4.302, including their assembly and linker-script inputs. It also compiles musl 1.2.5 and BusyBox 1.36.1. Both operating systems boot in QEMU: xv6 reaches its shell, and Linux starts the BusyBox init process and an interactive `ash` shell. The Linux build also runs file-system commands and reboots from userspace.

2. Structure

The compiler is one C program. The principal parts of `sccc.c` are tokenization and preprocessing, types and declarations, C expressions and statements, x86 instruction encoding, assembly parsing, ELF object input and output, symbol and archive handling, linker-script evaluation, section layout, and executable output. The output paths also write ELF32

objects, relocatable links, and shared ELF files. The code that produces BIOS and Multiboot images uses boot assembly from `lib/boot`.

`Ctx` carries the current tokens, sources, macros, scopes, types, symbol definitions, section buffers, and relocations. A single invocation can read several C and assembly translation units, followed by objects and archives. The driver also has a compile-only path. It writes one relocatable object per source input instead of running the executable linker when `-c` is given.

```
./sccc -c part.c
./sccc -c entry.S
./sccc main.c part.o entry.o -o program
```

In an ordinary hosted link, startup and the embedded library are added after the user inputs. In freestanding mode neither is inserted. The linker allocates common symbols, selects needed archive members, decides section addresses, resolves relocations, and writes the selected output format. The same section and symbol records are used by the compiler and the assembler. GNU ld scripts may replace the default section layout.

2.1. Buffers and allocation

`Buf` has three fields: `data`, `cap`, and `len`. Every append checks capacity, doubles it when necessary, and copies new bytes to the end. It is used for code, read-only literals, initialized data, token text, and the parts of the ELF file. `buf_patch32_at` changes a four-byte value already present in a buffer; branch patching uses it frequently. The small routines `le16`, `le32`, and `le64` write integers in the byte order required by ELF and x86-64.

Names are interned in a hash table of 8192 buckets. Each `Str` holds the spelling, length, keyword code, and an optional macro. Two occurrences of the same identifier refer to the same interned object. Name equality can therefore use pointer equality in the common parser paths. Symbols, types, members, parameters, macro expansion frames, and labels are individually allocated with `calloc` or `malloc`; their lifetime is the compilation process.

3. The language

The source language is C with the types and constructs implemented by the parser in `sccc.c`. Its basic types are `void`, `_Bool`, `char`, `short`, `int`, `long`, `float`, `double`, and `long double`, together with signed and unsigned integer forms and complex floating types. On the target, `char` and `_Bool` occupy one byte, `short` two, `int` four, and `long` eight. Pointers occupy eight bytes. `float` and `double` occupy four and eight bytes; `long double` uses the x87 80-bit extended format stored in a 16-byte slot. `signed char` and `long long` have type records of their own. They behave like `char` and `long` in ordinary integer code generation, but they remain distinct types for `_Generic` and `__builtin_types_compatible_p`.

`typedef`, enumerations, pointers, arrays, functions, structures, unions, and bit-fields are part of the parser. The recognized storage-class words include `static`, `extern`, and `typedef`. `auto`, `register`, and `inline` are read and discarded by `declspec`; `const`, `volatile`, and `restrict` are likewise accepted without a separate qualified type. Consequently, these spellings do not introduce an independent qualifier-enforcement or volatile-access mechanism in the generated code.

3.1. Type records and conversions

`Type` records the kind, size, alignment, signedness, base type, array length, members, parameters, and function attributes. Derived types are made by `ptr_to`, `array_of`, and `func_type`. The same type record is carried through expression parsing. For example, `a[i]` needs the size of the object pointed to by `a`; `p->field` needs the structure member's byte offset; and a call needs the declared parameter types.

Integer promotions are performed by `int_promote`; `arith_conv` selects the common arithmetic type when two operands differ. The resulting type controls extensions, signed or unsigned division, comparisons, and floating-point operations. A load of a signed `char` extends the sign bit into the result register, while a load of an `unsigned char` fills the upper bits with zeros. A cast to `_Bool` produces zero or one.

```
unsigned char byte = 255;
int x = byte;
unsigned long mask = 1UL << 40;
double y = (double)x / 2;
```

The value of `x` is 255. The shift uses the width of an unsigned long, and the division is floating point. These are separate machine-code cases. The type rules are therefore applied while parsing, not postponed until the executable is written.

3.2. Declarators and function types

C declarators can put a name inside several layers of pointers, arrays, and parentheses. The compiler splits the work between `declspec`, `declarator`, and `type_suffix`. `declspec` obtains the initial type and storage class; `declarator` reads pointers and a name, and `type_suffix` attaches array or function suffixes. A parenthesized declarator uses a temporary type node whose contents are filled after the outside suffix has been read.

```
int *a[4];           /* four pointers to int */
int (*f)(int);       /* pointer to a function */
int (*table[2])(int); /* array of function pointers */
```

Functions have parameter records and a result type. A declaration with an empty parameter list sets an `oldstyle` flag. A declaration with `void` gives an empty explicit parameter list. `...` sets the variadic flag. Array and function parameters are converted to pointers when their parameter types are formed. The parser checks argument counts for prototyped, non-variadic functions.

3.3. Pointers and arrays

An array type retains its element size and declared length. In value contexts an array yields its address; subscripting adds an index multiplied by the size of the element. The parser also accepts the reversed C form `i[a]`, since an integer and a pointer may appear on opposite sides of the subscript calculation. Pointer subtraction divides a byte difference by the pointed-to element size.

```
struct Pair { int left, right; };
struct Pair v[3];
struct Pair *p = v;
p[2].right = 17;
```

The address used by the store is `p + 2 * sizeof(struct Pair) + offsetof(right)`. The compiler gets both numbers from the type records. `&`, `*`, `.`, and `->` operate on addresses before `rval` converts an lvalue into its stored value. Assigning to an array or function is rejected; so is taking the address of a bit-field.

3.4. Structures, unions, and bit-fields

Structure members are laid out in declaration order with alignment between fields. The final structure size is rounded to its maximum member alignment. Union members start at offset zero, and the union size is rounded up from its largest member. A tag may first name an incomplete type and later supply its members. `Member` records contain names, types, offsets, and links to the following member.

```
struct Header {
    unsigned kind : 3;
    unsigned ready : 1;
    unsigned length : 12;
    char name[8];
};
```

Bit-fields are represented by their storage type, width, and bit position. The generator uses shifts and masks when reading or updating them. Signed fields are extended according to their width. An unnamed field can occupy bits without introducing a member name, and a zero-width field advances the packing position to the next storage unit. Anonymous structure and union members are folded into the outer member list with adjusted offsets.

A structure assignment copies bytes between addresses. The code generator uses the x86 string-copy instruction sequence `rep movsb` for this operation. Functions can return a structure, and the call mechanism provides storage for the returned object. These cases use the same size and member information that the declaration parser established.

3.5. Expressions

The expression parser uses precedence climbing. `binop_prec` gives the priority of arithmetic, comparison, bitwise, and logical operators. `parse_binary` reads the next tighter expression and then consumes operators of the requested precedence. Assignment is parsed separately, so it associates from right to left. The comma operator is handled by `parse_expr`, and the conditional operator by `parse_cond`.

```
int result = 2 + 3 * 4;
int flag = result > 10 && result < 20;
int chosen = flag ? result : 0;
```

Multiplication is emitted before addition. Comparisons produce integer truth values. `&&` and `||` use conditional jumps to skip the right operand when the left operand determines the result. This is a different code path from bitwise `&` and `|`, which evaluate both operands and combine their bits. The conditional operator also generates two control-flow paths and patches a jump where they meet.

The source handles prefix and postfix `++` and `--`, compound assignments, casts, `sizeof`, pointer arithmetic, and direct or indirect calls. Signed comparisons and unsigned comparisons select different condition codes. Integer division uses `idiv` for signed values and `div` for unsigned values. Floating-point arithmetic uses SSE instructions and separate integer/

float conversion routines. The bit pattern of a floating value is carried through ordinary expression storage in a general-purpose register; SSE registers are used while arithmetic or comparison is performed.

3.6. Long double and complex values

`long double` has its own type and the x87 extended format. The lexer turns decimal and hexadecimal long-double constants into ten-byte significand-and-exponent values. The conversion path uses the `Big` routines for multiword integer arithmetic and rounding, then places the resulting extended value in a sixteen-byte slot. A cast from `double` to `long double` loads a value onto the x87 register stack; a cast back stores it with the requested precision. The `gen_ldcast` routine handles integer, single-precision, double-precision, and extended-precision conversions. Ordinary `float` and `double` expressions continue to use SSE2.

```
long double x = 1.0L / 10.0L;
long double y = x * 10.0L;
int same = y == 1.0L;
```

The compiler uses x87 instructions for operations involving `long double`, and it saves intermediate values in stack slots when the expression requires an address. The return convention follows the x86-64 ABI: a scalar `long double` result is returned in `st(0)`. The type system also constructs complex floating types. A complex value consists of real and imaginary components, and the arithmetic generator handles the corresponding addition, subtraction, multiplication, division, and conversions. These operations are part of the same expression parser as the integer and SSE paths.

3.7. Variable-length arrays

An automatic array can take its length from a run-time expression. The size expression is evaluated where the declaration appears, multiplied by the element size, and rounded to the alignment used for the stack. The compiler adjusts `rsp` and records the resulting address in the variable's stack slot. `sizeof` on that array uses the size computed for the particular instance.

```
int checksum(int n) {
    int a[n];
    for (int i = 0; i < n; i++) a[i] = i * 3;
    int sum = 0;
    for (int i = 0; i < n; i++) sum += a[i];
    return sum;
}
```

A block saves the stack position at entry. Leaving the block restores it, including when control leaves through `break`, `continue`, `goto`, or `return`. `alloca` shares this dynamic stack allocation machinery. A kernel can therefore compile source that sizes temporary arrays from run-time values without changing the executable's data layout.

4. Preprocessing and tokenization

The lexer walks the source one character at a time. It skips spaces and both C comment forms, joins backslash-newline continuations, and tracks line numbers. It recognizes identifiers, numeric constants, strings, character constants, and punctuation, including multi-

character operators such as `>>=`, `->`, `&&`, and `##`. A token contains a kind, integer or floating value, text, source file and line, and flags for spacing and beginning-of-line status.

Preprocessing is integrated with token retrieval. A `Src` frame is either a source file with a lexer or a sequence of tokens produced by a macro expansion. Each frame points to its parent. `raw_next` obtains the next token from this stack; `expand_next` checks identifiers for macro expansion; `pp_next` passes the result to the parser. Includes and macro expansions use the same stack and naturally return to the previous input when exhausted.

4.1. Includes and conditionals

Quoted include names are searched relative to the including file, then through the `-I` paths. Angle-bracket includes use include paths and the built-in header table. Absolute include names can be read directly. `#pragma once` records a path so a later include of that path can be skipped. Include nesting is checked against a depth of 64 file frames.

The conditional directives are `#if`, `#ifdef`, `#ifndef`, `#elif`, `#else`, and `#endif`. The preprocessor maintains a conditional stack and skips inactive source groups without presenting their tokens to the C parser. `pp_eval` expands the tokens of a `#if` expression and uses the constant-expression parser to evaluate them. `defined` is handled while the expression is prepared. The compiler also recognizes `#undef`, `#error`, `#warning`, `#line`, and `#pragma`.

```
#ifndef BUFFER_SIZE
#define BUFFER_SIZE 1024
#endif
#if BUFFER_SIZE > 512
int large_buffer(void) { return 1; }
#endif
```

Conditional compilation has to run before declarations are analyzed: if a branch is inactive, the parser never sees its declarations or errors. Source file and line information follows the token through expansions so errors can be reported at their origin.

4.2. Macros

A `Macro` stores its name, parameter names, replacement tokens, and flags for function-like and variadic forms. The expansion routine collects actual arguments while counting nested parentheses. Arguments used normally are expanded before substitution. Arguments adjacent to `##` use their original token spellings. The `#` operator creates a string token, and `##` concatenates two spellings and lexes the result into a new token.

```
#define CAT(a, b) a ## b
#define STR(x) #x
#define TWICE(x) ((x) + (x))
int CAT(total, _count) = TWICE(4);
```

A macro that is currently being expanded is marked busy, preventing it from immediately expanding itself again. An empty macro argument may be represented temporarily by a placemarker token during concatenation. Variadic expansion supports `__VA_ARGS__`, including the comma-elision case used by the implementation. Predefined macros include `__FILE__`, `__LINE__`, `__STDC__`, platform names, and the target size constants. The builtin header and libc sources go through the same preprocessor as the user's files.

5. Parsing, names, and storage

The compiler keeps lexical scope with a linked list of `Sym` records and a nesting depth. `enter_scope` increments the depth. `leave_scope` drops the symbols introduced in that scope, exposing names that were shadowed. `Tag` records provide a separate list for structure and union tags. Enumeration constants use symbol entries with fixed integer values. A `typedef` name refers to a type rather than storage.

Global definitions have another representation. Their symbol entries include a section, an offset, a definition flag, and linkage information. `get_global` finds or creates such entries. A declaration seen in one file can therefore supply the target for a reference emitted in another file. Local variables instead use stack offsets recorded in their symbols.

5.1. Declarations are compiled as they are read

`parse_toplevel` reads one declaration at a time. A function declaration installs its type and name. A function definition continues into `parse_func_body`, which immediately emits the prologue, parameters, statements, and epilogue. A global initializer writes bytes to the data buffer or records a relocation if its value is an address. Declarations with no initializer normally reserve zero-initialized storage.

Inside a block, the parser recognizes declarations alongside statements. Automatic objects receive frame offsets through `alloc_local`. Their initializers generate instructions into the function body. The compiler also implements local `static` objects, which receive global storage while retaining the local name used to address them.

Consider a definition that comes after its caller:

```
int calculate(int);
int main(void) { return calculate(7); }
int calculate(int n) { return n * n; }
```

When `main` is parsed, `calculate` already has a function symbol, but not a code address. The call instruction receives a relocation record. When `calculate` is defined, the symbol records its position in the text section. `write_elf` uses that position to resolve the earlier call.

5.2. Initializers and constant expressions

Global initializers and local initializers are processed differently. For globals, integral and floating constants are converted into bytes in `.data`. An address initializer can write a placeholder together with an absolute relocation. For locals, the initializer evaluates at run time and stores through an address in the current stack frame. Array and structure initializers first clear the aggregate storage and then fill the specified fields.

The recursive initializer routines handle braces, string literals, arrays of unknown length, nested records, and designated initializers. An array without an explicit length obtains its length from its initializer. Designators such as `[3] =` or `.name =` select an element or member before parsing its value.

```
struct Item { int id; char code[4]; };
struct Item items[] = {
    [0] = { .id = 3, .code = "AB" },
    [2] = { .id = 9 }
};
```

The omitted array element is zero-initialized, and the unknown array length becomes three. The distinction between expressions executable at run time and expressions required to be constant appears here, in enumeration values, in `case` labels, and in array lengths. `const_expr` evaluates the integer form; separate routines evaluate floating constants for data initializers.

6. Machine-code generation

The backend writes x86-64 machine instructions directly. `Code` contains a byte buffer; the short routines beginning with `emit_` append opcode bytes, prefixes, ModR/M bytes, displacements, or immediate values. The small `C8`, `C32`, and `C64` functions are the final byte-writing operations. There is no textual assembly between expression parsing and the executable.

For example, loading a 64-bit constant into `RAX` writes the `REX.W` prefix, the immediate-move opcode, and eight bytes of data:

```
static void emit_mov_rax_imm64(Code *c, uint64_t imm) {
    C8(c, 0x48);
    C8(c, 0xB8);
    C64(c, imm);
}
```

There are analogous helpers for stack operations, integer arithmetic, comparisons, shifts, indirect calls, and floating-point instructions. The generated program is consequently built from a small collection of explicit encodings rather than from an instruction-selection table.

6.1. Values and addresses

The `lv` state tells whether an expression has produced an address. An identifier denoting a local variable emits its frame-relative address; a global variable emits a RIP-relative address with a relocation. `rval` then loads the object if a value is needed. Arrays yield pointers to their first elements, and a function designator yields a callable address. The same address machinery handles `&a`, `*p`, subscripts, and structure members.

```
int v = 10;
int *p = &v;
*p = *p + 1;
```

For the assignment, the left-hand dereference leaves a destination address. The compiler pushes it, computes the value on the right, restores the destination into another register, and chooses a store instruction using the size of the pointed-to type. A four-byte store and a one-byte store have different encodings; a bit-field store also reads the old storage unit, changes selected bits, and writes the unit back.

Most integer operations accumulate their result in `RAX`. For a two-operand expression the left value is pushed, the right value is computed, and the left is popped into `RCX`. Arithmetic then uses the pair of registers. `gen_push` and `gen_pop` maintain a count of temporary stack slots. The code generator uses it when making calls, because the stack must be aligned at the call site.

6.2. Floating-point instructions

The parser keeps the type of a floating expression even when its bytes reside in `RAX`. For an operation, helper routines transfer the bit pattern into SSE registers, perform the requested

instruction in single or double precision, and move the result back. Integer-to-float, float-to-integer, and float-to-float conversions have distinct paths. Floating comparison uses the SSE unordered comparison instructions and examines the condition flags, including the behavior of NaN.

The floating-point tests exercise mixed arithmetic, global constants, arrays of doubles, casts, comparisons, format printing, and mathematical functions from the embedded library. They also include a loop that adds `0.1` repeatedly, whose comparison with `1.0` shows the ordinary consequences of binary floating-point representation.

6.3. Branches and statements

A conditional branch can be emitted before its target has a position in the text buffer. `emit_jz_placeholder` and its relatives leave four zero bytes for a relative displacement. `patch_jmp_to` later replaces them with the difference between the target and the address following the instruction. Each `if` and `else` generates one or two such branches.

`while` records the condition position and emits a backward jump from the bottom of its body. `do` places its test after the body. `for` has condition, step, and body positions; the emitted code can jump over the step on the first iteration and return to it after the body. `break` and `continue` are stored in patch lists belonging to the active loop. A nested `switch` uses a separate kind of loop context so `continue` can still find the enclosing iteration statement.

`switch` first saves its controlling value. The case labels record positions in the body. After the body has been emitted, the compiler writes a chain of comparisons and jumps to those positions, with a final jump to `default` when present. `goto` names are stored in a function-local label list. A forward goto adds a patch to its label; defining the label resolves the pending jumps. Undefined and duplicate labels are diagnosed.

7. Constant folding and function expansion

Optimization happens while the parser still has the expression and its type. The `-O` family enables constant folding, the removal of code under constant conditions, the omission of statements after an unconditional transfer, and the expansion of selected inline functions. There is still one C parsing pass and no intermediate syntax tree. In a kernel build these local transformations also determine which sections and symbols reach the linker.

An integer constant is passed to `emit_const`, which chooses the encoding used to place it in `RAX`. `fold_binop` combines constant operands using the C conversion rules. When a local scalar has a known value, `kv_valid` and `kv` in its `Sym` record let `rval` substitute the constant for a load. Assignments, address-taking, calls, and control-flow boundaries invalidate or constrain that knowledge. The `kv_forget` routines implement those transitions.

```
static int scale(int x) { return x * 8; }

int next(int n) {
    if (0) return 99;
    return scale(n + 1);
}
```

With optimization enabled the dead branch is not emitted. Inline expansion can substitute the body of `scale` at the call site. `capture_body` keeps the function's token sequence, and `replay` reads those tokens again with the call arguments bound to parameters. `compile_lazy`

emits a retained static function when a live relocation requires it. `prune_statics` walks relocations between static data objects to retain the ones reachable from live code or exported data. These decisions are made from emitted code and symbol references rather than from an intermediate representation.

The `dead_begin`, `dead_end`, and `after_jump` routines stop instruction emission for paths excluded by constant control flow or by a `return`, `goto`, `break`, or `continue`. Labels referenced by live code restore emission at the necessary point. `-O0` keeps the direct, unoptimized translation; `-O1`, `-O2`, `-O3`, and `-Os` select the same compact set of local transformations. GNU inline attributes are carried on function symbols and affect whether their captured bodies are expanded or emitted as callable functions.

8. Functions and calls

A generated function saves `rbp`, establishes a frame pointer, and reserves stack space after the size of its locals and temporaries is known. Parameters received in registers are copied into frame locations. An epilogue restores the stack and returns. The code generator counts temporary pushes so calls are made with the stack aligned to 16 bytes.

8.1. System V argument classification

Calls use the System V x86-64 convention. Each scalar integer or pointer uses an INTEGER argument location. A `float` or `double` uses an SSE location. The six INTEGER registers are `RDI`, `RSI`, `RDX`, `RCX`, `R8`, and `R9`, and the eight SSE registers are `XMM0` through `XMM7`. The two register lists are allocated independently. Arguments left over after their registers are exhausted go on the stack. A scalar `long double` uses the x87 return register `st(0)`; the ordinary SSE register rules apply to `float` and `double`.

Structures and unions of up to sixteen bytes are classified one eight-byte unit at a time. The class of each unit depends on its members, with INTEGER taking precedence when integer and floating fields occupy the same unit. Large or unaligned aggregates are passed in memory. A register-returned aggregate uses the appropriate combination of `RAX`, `RDX`, `XMM0`, and `XMM1`. When a return value needs memory, the caller passes its address in `RDI`; the callee returns the address in `RAX`.

The compiler keeps the bit patterns of scalar floating-point values in `RAX` during ordinary expression evaluation. For a call or return it transfers the bits into or out of the prescribed SSE register. That movement is part of the calling convention. It permits an SCCC object to call a function from a GCC object and permits a GCC object to call one compiled by SCCC. The test suite exercises both directions.

8.2. Variadic functions and system calls

For a call to a variadic function, `AL` specifies the number of vector registers used. A variadic callee saves its incoming integer and SSE registers and initializes the System V `va_list` state, including offsets into its register-save area and the position of overflow arguments on the stack. `__builtin_va_start` and `__builtin_va_arg` implement the traversal. The calling convention applies to variadic functions compiled in separate object files as well as to the embedded `printf` implementation.

A Linux system call uses a different register arrangement. The built-in `__syscall` puts the call number in `RAX` and arguments in `RDI`, `RSI`, `RDX`, `R10`, `R8`, and `R9`, then emits `syscall`. That path does not use the ordinary C argument classifier.

9. The embedded C library

The files in `lib/include` define the public interfaces, and `lib/libc.c` implements them. `tools/embed.sh` converts these files into C string arrays in `builtin.h`. The host builds `sccc.c` with that header. When the resulting compiler encounters `#include <stdio.h>` or another embedded header, it supplies the corresponding stored text. When user compilation finishes, it compiles the embedded `libc.c` as another C translation unit unless `-nostdlib` was specified.

The library provides system-call wrappers, memory allocation, strings, character classification, file and formatted I/O, conversions, and mathematical routines. Its wrappers use the Linux syscall numbers for x86-64, including `read`, `write`, `open`, `close`, `lseek`, `mmap`, and `exit_group`. A negative syscall return value is converted to `-1` and stored in `errno` when appropriate.

The allocator obtains memory from `mmap`. Small allocations are grouped into size classes with free lists, and larger allocations obtain their own mapped blocks. `malloc`, `free`, and `realloc` are therefore available to programs compiled by SCCC. This matters for the compiler source itself: it makes extensive use of dynamically grown buffers and individually allocated symbols and types.

The I/O routines build on file descriptors. The library implements stream operations such as `fopen`, `fread`, `fwrite`, `fseek`, and `fprintf`; the tests use actual files as well as standard input and output. Formatted output covers integer formats and floating-point formats. The library is compiled by the same compiler as the program, so its functions follow the calling rules described in the preceding section.

A program can also be compiled with `-nostdlib`. In this mode, no library functions are added and no library startup call is emitted. The program must supply every function it references. The generated entry still calls `main` and terminates through the Linux exit syscall if no `exit` definition was supplied.

10. The ELF executable

Without a linker script, the hosted executable is an ELF64 `ET_EXEC` file loaded at `0x400000`. The output writer places the ELF and program headers with the compiled text, read-only objects, writable data, and zero-initialized data. Named sections from C attributes or assembly directives are also laid out. The file includes symbol and section-string tables so tools such as `readelf`, `objdump`, and `gdb` can examine it.

The linker places loadable code and read-only data in executable or read-only mappings, and initialized and uninitialized data in writable mappings. When thread-local objects exist, `.tdata` and `.tbss` are described by a `PT_TLS` segment. With a script, sections may have distinct virtual and load addresses, and the script can specify program headers explicitly.

10.1. Relocations and symbol resolution

Code emitted while parsing may refer to an undefined function, global variable, assembly label, or section. Each occurrence leaves a relocation with its section, offset, symbol, type,

and addend. A PC-relative relocation patches an instruction displacement; an absolute relocation patches an address. At the executable link, the writer looks up the final address and checks that the patched value fits its field.

For a four-byte RIP-relative field, the displacement is measured from the following instruction:

```
rel = target_address - (displacement_address + 4);
```

Relocatable ELF files keep such references in `.rela` sections for a later link. The supported x86-64 relocation kinds include `R_X86_64_PC32`, `R_X86_64_PLT32`, `R_X86_64_64`, `R_X86_64_32`, and `R_X86_64_32S`, with smaller forms from assembly and special forms for TLS and the global offset table. Weak definitions yield to strong ones, common definitions are merged, and undefined weak references have address zero. An unresolved strong reference is an error.

10.2. Process startup

For a hosted executable, `_start` gets `argc`, `argv`, and the environment from the initial Linux stack, aligns the stack, calls the embedded runtime initializer when present, and calls `main`. If the program has thread-local storage, the runtime establishes the thread pointer before user code needs it. The value returned by `main` becomes the process exit status. With `-ffreestanding`, startup belongs to the kernel source and the specified entry symbol instead.

11. Freestanding compilation

`-ffreestanding` changes how a compilation begins and ends. It sets `freestanding` and `nostdlib` in `Ctx`, so `main` is not required, `_start` is not generated, and the embedded C library is not compiled. The predefined macro `__STDC_HOSTED__` becomes 0. In this mode the program supplies its own entry function and whatever services it needs. A common entry is `kmain`, chosen with `-e kmain`.

```
./sccc -ffreestanding -mgeneral-regs-only \
  -oformat img -e kmain baremetal/hello.c -o hello.img
```

This command compiles an ordinary C source file. The C parser, type system, initialization rules, symbol table, and x86-64 emitter are the same ones used for Linux programs. The difference lies in startup and output. The entry symbol is resolved after all source files have been read, and the disk-image writer puts its address into the generated boot sector. The kernel does not enter through a C runtime; the boot code calls the selected function directly.

`-mgeneral-regs-only` makes a use of `float` or `double` an error. It is checked by `declspec` and by the floating-point expression paths, and it may be used with `-nostdlib` or `-ffreestanding`. This option is also required for interrupt functions. The generated kernel therefore uses the integer registers without depending on FPU or SSE state management during an interrupt. There is no new dialect of C for the kernel: pointers, structures, arrays, function calls, and control flow use the same compiler machinery.

Three options determine kernel placement. `-oformat` chooses `elf`, `bin`, `img`, or `multiboot`; `-Ttext` selects the address at which `.text` executes; `-e` names the entry function. The default text address for a BIOS image is `0x100000`, and the default for a Multiboot image is `0x101000`. The corresponding addresses are absolute runtime addresses, not offsets in the disk file. For a flat binary, the default text address is zero, and its entry must be the first byte of the text. The writer checks that condition.

12. Flat binaries and boot images

12.1. Section layout

The ordinary output writer and the linker-script path both assign addresses for the supported output formats. `.text` contains emitted instructions, `.rodata` contains strings and other read-only constants, `.data` holds initialized objects, and `.bss` reserves zero-filled memory. A section named in a `section` attribute follows the built-in section of its kind. The writer first builds the order: `.text` and the code sections, `.rodata` and the read-only sections, `.data` and the data sections, then `.bss`. It then gives each one an address aligned to the largest alignment requested inside it. References between these sections are resolved before the output bytes are written. In the flat format the sections other than `.bss` appear consecutively with alignment padding. `.bss` occupies no file bytes; its extent is included in the address calculations used by the boot loaders.

```
./sccc -ffreestanding -oformat bin -Ttext 0x100000 \
  -e _start kernel.c -o kernel.bin
```

Here `kernel.c` defines `_start` as its first function. In the flat case `-Ttext` describes where the loader will place the first instruction. The file contains no ELF header and no loader of its own. A kernel using a later entry address is assembled by choosing an output format that records that address. `-oformat img` and `-oformat multiboot` both place their own startup code before the kernel and arrange to call the selected entry point.

12.2. The BIOS disk image

`write_image` prepends a 512-byte boot sector to the kernel image. The boot sector is assembled from `lib/boot/img.s` using the compiler's own assembler, with kernel placement values supplied at image construction time. Its execution begins at `0x7c00` in the mode established by the PC BIOS. The final two bytes of the sector are `0x55` and `0xaa`, the BIOS boot signature. The kernel itself starts at the next disk sector.

Before loading the kernel, the boot sector requests the BIOS E820 memory map with `int 0x15`. It stores the entry count at physical address `0x5000`, followed by 24-byte entries beginning at `0x5008`. Each entry gives a base address, a length, a type, and extended attributes. This table is passed to C code later; the `memory` kernel reads it to decide which physical page frames are available.

The sector reads the kernel from disk with the extended `int 0x13` interface, using a disk address packet. The initial load address is `0x10000`, below the 1 MB boundary where the firmware interface can reach it. Reads proceed in batches of at most 64 sectors. A disk-read failure prints an `E` through the BIOS video interrupt and halts. The image writer measures the kernel, rounds its file size to whole sectors, and records the count in the boot sector. It rejects kernels larger than 588800 bytes.

After the read, the sector enables the A20 line through port `0x92` and constructs page tables at physical addresses `0x1000` through `0x3fff`. A PML4 entry points to a page-directory-pointer table, whose first entry points to a page directory containing 512 entries of 2 MB each. The result is an identity mapping of the first gigabyte. The setup loads `cr3`, enables PAE in `cr4`, sets the long-mode enable bit in `EFER`, loads a GDT, and enables paging in `cr0`. A far jump completes the transition into 64-bit long mode.

Now the boot code copies the kernel from its temporary load buffer to the `-Ttext` address, clears `.bss`, sets `rsp` to `0x80000`, places `0x5000` in `rdi`, and calls the entry function. The C entry therefore receives a pointer to the memory-map record without calling the BIOS itself. The loader leaves interrupts disabled. The kernel decides when to construct an IDT and enable external interrupts.

The image is padded to a cylinder-sized multiple of 516096 bytes, using 16 heads and 63 sectors per track. The boot image is therefore a complete raw PC disk image. QEMU and Bochs boot it through their BIOS disk paths.

12.3. The Multiboot image

The Multiboot output has a different first page. `write_multiboot` assembles the trampoline from `lib/boot/multiboot.s` and writes the Multiboot 1 magic value `0x1badb002`, flags, checksum, and explicit load addresses, followed by a trampoline that starts in 32-bit protected mode. The next page begins the x86-64 kernel at the text address, normally `0x101000`. The text address must be page aligned. The image can be loaded by GRUB or by QEMU's `-kernel` option.

The Multiboot trampoline receives the bootloader's registers and saves them while it constructs identity-mapped page tables. Its page tables and a 16 KB stack occupy space after the kernel's BSS. It sets `cr3`, `cr4`, `EFER`, and `cr0`, loads a small GDT, and jumps into long mode. It then passes the address of the Multiboot information structure in `rdi` and the magic value `0x2badb002` in `rsi`, and calls the C entry. The BIOS image and the Multiboot image reach the same kind of 64-bit kernel function by different boot paths.

```
./sccc -ffreestanding -mgeneral-regs-only \
  -oformat multiboot -e kmain baremetal/hello.c -o hello.mb
qemu-system-x86_64 -kernel hello.mb -serial stdio
```

The Multiboot header and long-mode trampoline are assembled by SCCC from the boot sources embedded with the compiler. No external assembler or linker is run when these images are constructed.

13. Machine instructions and interrupts

13.1. Hardware built-ins

The compiler recognizes machine operations by name while parsing a call. Their encodings are described in `mach_builtins`, an array of records containing the name, argument count, argument registers, result type, instruction length, and bytes to emit. Arguments are parsed as normal C expressions, checked as integers or pointers, and moved into the required registers. The emitter then appends the bytes. For example, `__outb` places the port in `dx`, the value in `al`, and emits opcode `0xee`. `__inb` emits `in al, dx` followed by an extension that produces an unsigned-byte result in `eax`.

```
__outb(0x3F8, 'A');
unsigned char status = __inb(0x3FD);
__cli();
__lidt(descriptor);
__sti();
__hlt();
```

The built-ins cover byte, word, and double-word port I/O; `cli`, `sti`, `hlt`, and `pause`; loading the GDT and IDT; loading the task register; reading and writing `cr0`, `cr2`, `cr3`, and `cr4`; reading and writing model-specific registers; invalidating a TLB entry with `invlpg`; querying `cpuid`; and reading `rdtsc`. The `__cpuid` operation stores four 32-bit registers into the caller's result array. `__rdmsr` and `__rdtsc` combine `edx:eax` into a 64-bit C result.

There is also a distinct `__syscall` built-in for Linux, whose arguments follow the Linux syscall register order. Kernel examples use the hardware built-ins to reach I/O ports and control registers, without relying on inline assembly. The compiler emits precisely the processor operations named in the source.

13.2. Interrupt functions

An interrupt gate cannot enter a function through an ordinary C `call`. The processor has already pushed an interrupt-return frame, and the exit instruction must be `iretq`, not `ret`. A fault that supplies an error code also changes the location of that frame. SCCC handles both cases with the `interrupt` function attribute.

```
struct frame {
    unsigned long ip, cs, flags, sp, ss;
};

__attribute__((interrupt))
void on_timer(struct frame *f) {
    (void)f;
    ticks++;
    __outb(0x20, 0x20);
}
```

`emit_interrupt_entry` writes a wrapper at the beginning of an attributed function. It first clears the direction flag, then saves `rax`, `rcx`, `rdx`, `rsi`, `rdi`, and `r8` through `r11`. It aligns the stack for a call into the regular C function body and passes a pointer to the processor's interrupt frame as the first argument. If the handler has a second parameter, the wrapper reads the hardware error code and passes it as an `unsigned long`. On return it restores the saved registers, removes the error code where present, and executes `iretq`.

The compiler checks the function signature: an interrupt function returns `void` and has either a frame-pointer argument or a frame pointer followed by one 64-bit integer error-code argument. It also requires `-mgeneral-regs-only`. The ordinary function prologue and body follow the generated wrapper, so an interrupt handler can use local variables, C expressions, and calls to other C functions. `emit_interrupt_entry` makes the ABI boundary explicit in the generated instructions.

13.3. Descriptor tables and devices

`baremetal/kernel.h` supplies the small amount of code shared by the example kernels. It writes a 16-byte IDT gate as two 64-bit words, constructs a 10-byte IDTR descriptor, and loads it with `__lidt`. `pic_init` remaps the master and slave 8259 interrupt controllers, normally placing hardware IRQs at vectors 32 and above. The same header initializes the first serial port at `0x3f8`, waits for transmitter status at `0x3fd`, and prints characters with `__outb`.

The header also contains a `finish` routine. It waits for serial output to drain, writes `0x10` to port `0xf4` for QEMU's ISA debug-exit device, and sends `Shutdown` to Bochs through port

`0x8900`. These ports provide a programmatic end to kernel tests. The remaining code path disables interrupts and halts.

14. Kernel programs

The `baremetal` directory contains seven kernel programs written in C. Each uses the disk image output, starts through the generated boot sector, and runs without the Linux runtime. Their output goes through COM1; `hello` also writes directly to VGA text memory at physical address `0xb8000`.

The `hello` kernel initializes the serial port, writes a line of text, fills the 80-by-25 VGA character cells, and verifies that the character-and-attribute words can be read back. It prints the result over serial before stopping the emulator. The kernel does not call `printf`. Its output routine is a loop over I/O-port writes in `kernel.h`.

The `timer` kernel constructs an IDT entry for vector 32, reprograms the PIC, and sets the programmable interval timer by writing its control word to port `0x43` and the divisor to port `0x40`. It enables interrupts with `__sti` and halts between ticks with `__hlt`. The interrupt function increments a counter and acknowledges the PIC by writing `0x20` to port `0x20`. The main loop prints every tenth tick up to 50. It also reads the timestamp counter before and after the run.

The `exceptions` kernel installs handlers for divide error (#DE), breakpoint (#BP), and page fault (#PF). It enters the breakpoint vector with a byte sequence containing `int3`, then performs an integer division by zero and changes the saved instruction pointer in the exception frame to resume execution. The page-fault handler reads `cr2` to find the faulting address, examines `cr3` to locate the page tables, inserts a second identity-mapped gigabyte, and invalidates the faulting translation with `__invlpg`. The test checks both the supplied error code and the address that caused the fault. It also verifies that a local 64-bit value survives the exception sequence.

The `keyboard` kernel handles PS/2 keyboard interrupts on vector 33. Its handler reads a scancode from port `0x60`, puts it into a 64-entry ring buffer, and acknowledges the PIC. The main loop consumes the buffer, tracks Shift, converts key presses through a table, and echoes characters over serial until Enter. The QEMU test sends keys through its monitor. The Bochs build defines `INJECT` and places the corresponding scancodes into the controller through ports `0x64` and `0x60`.

The `memory` kernel receives the E820 map passed by the BIOS boot sector. It creates a bitmap covering the first gigabyte in 4096-byte page frames, marks only usable memory above 2 MB available, and allocates and releases frames while checking their contents. It also finds a consecutive 256-frame run and uses it as a one-megabyte heap. The heap keeps a linked list of blocks, splits a block for allocation, and coalesces adjacent free blocks on release. A test performs 3000 pseudorandom allocations and frees, checking data and 16-byte alignment. Both the page-frame and heap tests finish with their free counts restored.

The `tasks` kernel adds interrupt stubs and context switches written in a separate `.S` file. It installs handlers for breakpoint, division error, invalid opcode, and general protection faults. A timer interrupt enters through the IDT, three tasks switch through saved interrupt frames using `int $0x80`, and a coroutine changes context through `switch_to`. Assembly calls back

into C through a table of function pointers. This program exercises the connection between C functions, interrupt frames, assembler labels, and kernel stack layouts.

The `initcall` kernel is laid out the way larger kernels organize their start-up code. Each init function is registered with a macro that places a function pointer in the section `initcalls`, and `kmain` walks the table from `__start_initcalls` to `__stop_initcalls`. One init function builds new page tables in static arrays declared with `aligned(4096)`. They identity map the first gigabyte and map it a second time at `0xffffffff80000000`. The function loads them with `__write_cr3` and checks a variable through both addresses. Another runs a small bytecode program through a table of label addresses and `goto *`. A third classifies characters with case ranges and fills a table with range designators. The last checks that an undefined weak symbol has address 0 and that an alias calls its target. Output goes through `_Generic` and statement-expression macros.

The seven programs exercise different parts of the compiler while running on the processor directly: volatile memory accesses, integer types and pointer arithmetic, hardware operations, function wrappers, control-flow patching, constant data, global initialization, and structure layouts. The `baremetal/run.sh` suite compiles each kernel, boots it in QEMU and Bochs, and compares its serial output with the checked-in `.out` file. QEMU completion is also checked through the debug-exit status; Bochs completion is checked through its shutdown request. The source of the kernels and their expected output live together in the repository.

```
make baremetal
# or, as part of the complete test suite:
make check
```

The boot sector and Multiboot trampoline come from assembly source embedded with the compiler and assembled by SCCC. The hardware built-ins and interrupt wrappers are generated from C. Both paths use the same x86 instruction encoder.

15. Compiling xv6

The xv6-x86_64 tree is built with SCCC as its C compiler, assembler, and linker. Its Makefile invokes the same executable for all three operations. Kernel C files compile to relocatable objects; the assembly sources provide entry code and low-level processor operations. The final kernel link uses `kernel64.ld` to assign the kernel sections and produce the ELF image loaded by QEMU. The user programs are also compiled and linked, and their binaries are put in the xv6 file-system image.

```
make xv6run
```

`tools/xv6run.sh` obtains the tested xv6 revision when needed and builds `out/kernel.elf` and `fs.img` through xv6′s Makefile, passing the SCCC path as `CC`, `AS`, and `LD`. QEMU starts with two CPUs and the prepared file-system disk. On a terminal the kernel starts its processors, runs `init`, and presents the xv6 shell. From that shell, `ls`, `cat`, `grep`, and the other installed user programs execute as xv6 processes.

```
xv6...
cpu0: starting xv6
cpu1: starting
init: starting sh
$ ls
```

```
.               1 1 512
..              1 1 512
README          2 2 1588
cat             2 3 15664
sh              2 13 26512
```

The console excerpt is from the xv6 run built with SCCC. Reaching the shell requires successful compilation and linking of the kernel's process scheduler, virtual-memory code, interrupt paths, file system, and system-call interface. It also requires the separate user programs and their syscall stubs to have the correct entry points and calling conventions. A compiler that merely produces a bootable kernel would not satisfy those dependencies.

The `make xv6` test performs a longer check. `tools/xv6.sh` builds xv6 with SCCC and builds a GCC/GNU-toolchain reference, boots both with QEMU, runs xv6 `usertests`, and compares normalized serial output. `make xv6run` is the interactive route; `make xv6` is the automated operating-system test. The xv6 source is kept separately from the compiler sources. The observed shell boot and the test driver use the same compilation and linking paths.

16. Compiling Linux

The Linux build uses the compiler as the C compiler, assembler, and linker under Kbuild. `tools/linuxrun.sh` obtains Linux 5.4.302 from kernel.org, checks its checksum, applies `tools/linux.config` to a `tinyconfig`, and creates an out-of-tree build under `linux/build-sccc`. The selected configuration enables x86-64, the PVH entry, serial console output, an initial RAM filesystem, and ELF program loading. The build produces `vmlinux` using `sccc` for `CC`, `AS`, and `LD`; Kbuild's host-side utilities are built with `HOSTCC=gcc`.

```
make linuxrun
# or run the build stages separately:
tools/linuxrun.sh fetch
tools/linuxrun.sh config
tools/linuxrun.sh build
tools/linuxrun.sh init
```

Kbuild supplies C sources, preprocessed assembly, relocatable objects, archives, linker command lines, and the `vmlinux` linker script. Linux also uses GNU C syntax, attributes, inline assembly, local and global symbol bindings, and generated headers. SCCC reads the same files and places the resulting sections into the kernel image. The build therefore exercises the compiler as an independent toolchain through a considerably larger translation workload than its own source.

16.1. The first user process

The Linux test has two boot stages. The first runs the kernel without an initramfs. Serial output reaches the missing-init panic after the kernel has initialized its subsystems. The second stage supplies an initramfs containing a statically compiled `/init` from `tools/linuxinit.c`. The script creates the archive with the kernel's `gen_init_cpio` tool, then passes it to QEMU with `-initrd`.

```
void _start(void) {
    put("sccc init: hello from userspace\n");
    put("sccc init: pid ");
```

```
    putnum(sys(39, 0, 0, 0, 0));
    put("\n");
    sys(169, 0xfee1dead, 672274793, 0x01234567, 0);
    for (;;) sys(34, 0, 0, 0, 0);
}
```

The program starts without the hosted C runtime and issues Linux system calls through inline assembly. It writes to the serial console and obtains its PID from the running kernel. The printed message establishes that control has passed from the kernel's ELF loader into an SCCC-compiled user program. QEMU boots the PVH kernel directly with `-kernel`; when KVM is available the script enables it.

16.2. Debugging the kernel build

Kernel object files are useful units of comparison because GCC and SCCC produce the same ELF object format and use the same external calling convention. During the Linux bring-up, I substituted GCC-built objects for SCCC-built ones until the boot failure narrowed to one object. Comparing its machine code and relocations with the GCC output then exposed the offending generation error. The final kernel links objects from both compilers in the same program during this diagnostic process.

This method separates the parser and code generator from the linker and boot environment. A kernel that works with a single GCC object substituted identifies the source unit to inspect; an instruction or relocation comparison can then be made around the failing function. The mixed-object ABI and ELF tests already used by SCCC serve the same purpose at a smaller scale.

17. Building musl and BusyBox

The kernel boot is followed by a userspace built with SCCC. `tools/busyboxrun.sh` obtains musl 1.2.5 and BusyBox 1.36.1, checks their release archives, and creates the runtime and shell used by the kernel. It uses the kernel's installed UAPI headers for types and system interfaces and keeps the runtime, application, and kernel builds in separate directories.

17.1. The C library

musl is configured with SCCC as its C compiler and built into a static `libc.a`. SCCC compiles the library's C and assembly sources, leaving the normal archive creation to `ar` and `ranlib`. The resulting library provides the C startup files and functions that a separate application expects from a Linux C library. It is distinct from SCCC's embedded library, which remains the runtime used for an ordinary single-command compilation.

The `tools/musl-sccc` wrapper supplies musl's include path and the kernel UAPI headers for compilation. At link time it uses `-nostdlib`, adds `crt1.o` and `crti.o` ahead of the user inputs, then places `libc.a` and `crtn.o` after them. In this way the application uses musl's startup and library, while SCCC emits the object files and performs the link.

```
# The wrapper is used by the BusyBox build as CC:
tools/musl-sccc -c example.c -o example.o
tools/musl-sccc example.o -o example
```

17.2. BusyBox and the initramfs

BusyBox is configured from `allnoconfig` and `tools/busybox.config`, then compiled with `tools/musl-sccc`. Its applets share one executable and are reached through symbolic links under `/bin`, `/sbin`, and `/usr`. The initramfs also holds `/etc/inittab` and `/etc/init.d/rcS`. The startup script mounts `proc`, `sysfs`, `devtmpfs`, and `tmpfs` before starting an interactive shell.

```
make busyboxrun
make busyboxcheck
```

`busyboxrun` builds Linux with the configuration in `tools/linux-busybox.config` added to the basic kernel configuration. It enables the filesystem and process interfaces needed by the shell, including procfs, sysfs, tmpfs, devtmpfs, futexes, and the usual event and timer facilities. QEMU loads the kernel with the initramfs and starts BusyBox as `/init`. The shell can run applets from its own executable, use pipes and redirections, create background jobs, read `/proc`, and power off the virtual machine.

The automated session runs `uname -a`, `ls /`, `cat /proc/meminfo`, `ps`, `mount`, shell arithmetic, a pipe, a here-document, a background process, and a nonexistent command whose exit status is checked. The final command powers off the guest. `make busyboxcheck` also builds musl and BusyBox with GCC, runs `tools/musltest.c` and `tools/bbtest.sh` against both toolchains, and compares the outputs. The comparison includes formatted I/O, command substitution, shell functions, file operations, sorting, text utilities, and process handling.

17.3. Booting Linux 5.4.302

The `make busyboxrun` target builds musl 1.2.5, installs the Linux UAPI headers, compiles BusyBox 1.36.1, and then builds Linux 5.4.302 with SCCC. `tools/busyboxrun.sh` assembles the initramfs and starts the kernel in QEMU with the serial console on `ttyS0`. The kernel identifies its compiler as `sccc version 1.0 (x86-64)` and proceeds through memory initialization, ACPI and PCI discovery, interrupt-controller setup, and serial-device initialization before launching `/init`.

The following lines are from the interactive `make busyboxrun` session:

```
Linux version 5.4.302 (root@v4n) (sccc version 1.0 (x86-64))
Console: colour dummy device 80x25
printk: console [ttyS0] enabled
Unpacking initramfs...
Run /init as init process
init started: BusyBox v1.36.1
sccc busybox: system ready
BusyBox v1.36.1 built-in shell (ash)
~ # ls
bin   dev   etc   init  proc  root  sbin  sys   tmp   usr
~ # pwd
/
~ # reboot
The system is going down NOW!
Requesting system reboot
reboot: Restarting system
```

The shell is the BusyBox executable linked against the musl library compiled by SCCC. Its startup script has mounted the virtual filesystems and started the shell from the initramfs. `ls` reads the root directory, `pwd` reports `/`, and `reboot` initiates an orderly shutdown through the Linux userspace and kernel interfaces. The QEMU session ends after the guest restarts. This run exercises C compilation, x86 assembly, ELF object and archive linking, the kernel linker script, musl startup code, process creation, system calls, the initramfs, and the interactive shell in one build.

18. Multiple source files

Each C file has its own preprocessing run and file-scope names. Definitions that need external linkage become global symbols in the output. Assembly labels declared with `.globl` participate in that same symbol namespace. SCCC can therefore compile several C and assembly files at once or compile them separately and link the results later.

```
./sccc -c main.c
./sccc -c part.c
./sccc main.o part.o -o program
```

The linker accepts relocatable `.o` files and static `.a` archives from other tools as well as from SCCC. An archive is scanned for members that define currently unresolved symbols. Extraction repeats until the archive has no further needed members. The order of archives on the command line affects which definitions are seen. The embedded runtime library is processed after the user inputs unless `-nostdlib` is specified.

The same ABI permits a GCC-compiled object to be linked into an SCCC executable and an SCCC-compiled object to be linked by GCC. External objects that refer to unsupported libraries or runtime facilities still require those implementations to be supplied. Here the object boundary is real: individual C and assembly sources need not be recompiled when another file changes.

19. Assembling x86 source

The assembler reads GNU AT&T syntax. Register operands start with `%`, immediates with `$`, and a memory operand has the form `displacement(base, index, scale)`. The source operand precedes the destination. `.s` files are assembled directly; `.S` files pass through the C preprocessor with `__ASSEMBLER__` defined. A preprocessed assembly header can therefore hide C declarations behind a conditional directive.

```
.globl increment
increment:
    leal 1(%rdi), %eax
    ret
```

Instructions are encoded through the same routines the C generator uses: prefixes, opcode bytes, ModRM and SIB fields, displacements, and immediates. `.code16`, `.code32`, and `.code64` change the instruction mode, which matters to boot code that passes from real mode through protected mode into long mode. Numeric labels such as `1f` and `1b` are resolved within an assembly unit; named local labels and global labels have separate visibility rules.

19.1. Sections, directives, and branch sizes

Directives select `.text`, `.rodata`, `.data`, `.bss`, or a named section. Data directives write bytes and words, string directives append text, `.zero` reserves space, and alignment directives pad the current section. Symbol directives define globals, weak names, types, sizes, and aliases. The assembler also accepts conditional assembly and inclusion directives used by the kernel and xv6 sources.

An assembly unit is read in passes. A forward branch initially may not know whether its target fits an eight-bit displacement. If it does not fit, the next pass uses the longer branch form. The passes repeat until no further branch grows. Unresolved cross-section and external references become relocations for the linker. Assembly and C are thus joined at the section and symbol level, not by textual inclusion.

The source test `tools/ascorpus.py` compares instruction encodings with an external assembler over a generated corpus. The mechanism being tested is the shared encoder, so this also exercises instruction forms used by the C compiler.

19.2. Assembler macros and repetitions

The assembly front end also has macro expansion. `.macro` records a body and its formal parameters; invocation substitutes the arguments and pushes the resulting text back into the assembler input. `as_macro_def`, `as_macro_call`, and `as_subst` do the recording and substitution. The assembler uses the same stack of text inputs for `.rept`, `.irp`, and `.irpc`, so a repeated instruction block is assembled by the ordinary instruction parser after substitution.

```
.macro save_reg reg
    pushq \reg
.endm
save_reg %rbx
```

An assembly macro is expanded before instruction encoding. Numeric local labels, section changes, and relocation expressions in its body are then handled by the existing assembler passes. This is used with the assembly fragments and generated sources encountered in kernel builds.

20. Inline assembly

Basic `asm` inserts assembly source at the point of the C statement. Extended assembly supplies inputs, outputs, and clobbers. A template may refer to operands by position or by a name in brackets. The parser assigns general registers, constants, or memory locations according to constraints, expands the template, and passes it to the internal assembler.

```
static unsigned long read_flags(void) {
    unsigned long flags;
    __asm__ volatile("pushfq; popq %0" : "=r"(flags));
    return flags;
}
```

The constraint `=r` means that the output receives a general-purpose register. Input constraints may require particular registers or permit immediate and memory operands. A matching constraint reuses an output location; an early-clobber constraint keeps an output separate from inputs. Clobbered callee-saved registers are preserved around the statement. A `memory` clobber is recognized, and an `asm goto` template may transfer control to a C label.

The compiler also accepts a register name on a local declaration with `asm("r10")`, which is used for operands such as the fourth argument of the Linux `syscall` instruction. Inline assembly and external `.s` files go through the same parser and encoder, with different handling of their source locations and C operands.

21. Relocatable ELF objects and archives

The `-c` path writes `ET_REL` ELF64 objects. Alongside section contents, each file contains `.symtab`, `.strtab`, `.shstrtab`, relocation sections of type `SHT_RELA`, and a `.note.GNU-stack` section. The writer records section-relative values rather than final virtual addresses. Calls normally use `R_X86_64_PLT32`; other RIP-relative references use `R_X86_64_PC32`. Pointer initializers use absolute relocations of the appropriate width.

For ordinary links, the object loader reads section headers, copies loadable contents to internal sections, imports symbols, and records the relocations. Static symbols are scoped to their object. Global symbols enter the linker's name table. Common symbols from tentative definitions are merged, while weak definitions can be replaced by strong definitions. The loader recognizes the relocation forms used by the compiler, assembler, and the supported GCC object cases.

Archives have a different rule. The whole archive is not copied into the link. Its members are inspected and extracted when they define an outstanding reference. The process repeats within an archive because one extracted member may introduce another reference. This is why file order matters for archives and why a `-c` build needs no startup code or embedded libc until the final executable is linked.

21.1. Global offset table and thread-local relocations

The ELF reader also handles GOT-relative references from supported position-independent input objects. A `.got` section receives slots for the symbols that need them, and the linker fills those slots with the final addresses. The output remains a statically linked, position-dependent executable, but the input object may use GOT addressing.

Thread-local objects require another layout. Initial thread data goes in `.tdata`; zero-initialized thread data goes in `.tbss`. The linker computes offsets from the thread pointer and resolves local-exec relocations such as `R_X86_64_TPOFF32`. Their runtime address is obtained through the FS segment rather than an ordinary global address. Hosted startup initializes that segment from the ELF thread-local storage description; freestanding programs establish it themselves.

22. Linker scripts

The `-T` option reads the GNU ld script used to lay out the output. It can name the entry point, define symbols, place input sections in output sections, set alignments, and give sections different virtual and load addresses. A bare-metal kernel commonly needs all of those operations.

```
ENTRY(boot32)
KERNEL_VMA = 0xffffffff80000000;
SECTIONS
{
  . = 1M;
```

```
    .boot : { KEEP(*(.multiboot)) *(.boot.text) }
    . += KERNEL_VMA;
    .text ALIGN(4K) : AT(ADDR(.text) - KERNEL_VMA)
         { *(.text .text.*) }
    .data : { *(.data) }
    .bss  : { *(.bss) *(COMMON) }
  }
```

The example follows the kernel script in `baremetal/higher`. The boot code is loaded at a low physical address. The kernel text is linked at a high virtual address, with `AT` computing its physical load address. The script's location counter, `.`, advances independently of the bytes written to the file. The linker uses those values when resolving references between the boot code, text, and data.

`SECTIONS` contains output-section declarations and assignments. Wildcard patterns select input sections by name or input file. `KEEP` preserves selected input; `/DISCARD/` drops selected sections. The implementation supports `SORT` forms, common linker-expression operators, `ALIGN`, `ADDR`, `LOADADDR`, `SIZEOF`, `DEFINED`, `PROVIDE`, and `ASSERT`, as well as program-header descriptions through `PHDRS`. The script may request ELF or binary output.

Section placement is iterative where an expression refers to an address not yet known. Each input section is assigned to its first matching output-section rule. Unmatched sections are placed as orphans according to their kind. The final program headers group sections with compatible load mappings. For binary output the writer emits the loadable bytes starting at the lowest load address, omitting uninitialized storage. The image writers can then add the appropriate BIOS or Multiboot prefix.

The tests in `tests/ldscript/` exercise executable, flat, higher-half, and orphan-section layouts. `tools/ldfuzz.py` compares additional generated script cases. This parser is part of SCCC's executable linker; it does not invoke GNU ld to interpret the script.

23. The compiler driver for kernel builds

Kbuild treats the compiler, assembler, and linker as separate programs and passes the options used by GNU tools. SCCC accepts those commands through one driver. `-E` preprocesses a source unit, `-x` specifies its language, `-include` injects a header before it, and `-isystem`, `-iquote`, and `-idirafter` select include search directories. `-std=gnu89`, `gnu99`, and `gnu11` set the GNU language mode and the corresponding `extern inline` rules. The driver defines the GNU compatibility macros used by kernel headers. The dependency options `-M`, `-MM`, `-MF`, `-MT`, `-MQ`, and `-MP` are processed by `write_deps`.

The linker path accepts archives as inputs and implements `--whole-archive`, `--no-whole-archive`, `--start-group`, and `--end-group` to control member extraction. Other Kbuild link options select section collection, retained relocation information, GNU symbol versions, a build ID, and an ELF hash table. Input options such as `-z max-page-size=` are passed to the layout machinery. The command-line parser builds the link context used by `link_inputs` and the ELF writers.

The important output switches select a kind of file, not a separate external tool: `-c` writes objects, `-r` performs an intermediate relocatable link, `-shared` writes ELF shared objects, and `-T` lays out sections under a script. This is how Kbuild's compile, assemble, intermediate link, vDSO link, and final `vmlinux` link enter SCCC through the same executable.

24. Kernel object formats and shared ELF

The Linux build exercises three ELF output paths alongside the ordinary x86-64 executable writer. Early x86 boot code uses 16-bit and 32-bit instructions and ELF32 objects; later kernel code uses ELF64 relocatable objects. The link of `vmlinux` uses a GNU ld script, and the vDSO is constructed as an ELF shared object. Each path retains the same symbol and relocation machinery, with an ELF header and relocation representation for the selected class.

24.1. ELF32 boot objects

The assembler understands `.code16`, `.code32`, and `.code64`. Command-line `-m16` and `-m32` select the starting assembly mode. The linker emulation `-m elf_i386` writes ELF32 from these objects. `elf_to32` writes the ELF32 headers and relocations after the relevant addresses and offsets have been assigned. `rel386` handles the relocation forms used by the 32-bit link. Kbuild uses this path for real-mode boot components before the processor transitions to long mode.

```
./sccc -m32 -c boot.s -o boot.o
./sccc -m elf_i386 -T boot.ld boot.o -o boot.elf
```

The assembler may change mode within one source file. The C compiler continues to generate x86-64 code; the 16-bit and 32-bit paths belong to the assembler and ELF writer. Linux therefore reaches its 64-bit C code through startup instructions that have been assembled and linked by the same program.

24.2. Relocatable links and the vDSO

The `-r` option links several inputs into a new relocatable object. Defined symbols and input sections are combined, while relocations that still require a final address remain in the output. This corresponds to the intermediate links performed by Kbuild between directories and at later kernel build stages. Archive extraction and weak-symbol rules apply at the same point in the link.

The `-shared` path writes ELF type `ET_DYN`. It builds `.dynsym`, `.dynstr`, `.hash`, `.gnu.hash`, and `.dynamic`, using the functions `elf_hash`, `gnu_hash`, `ld_shared_prep`, and `ld_shared_fill`. The script interpreter reads `VERSION` blocks through `ld_version` and assigns the symbol versions used in a Linux vDSO. The vDSO is the small kernel-provided shared ELF image mapped into each process so user code can call selected kernel services without a conventional system call on every invocation.

GNU ld script support now also assigns program headers through `PHDRS` and `ld_resolve_phdrs`. Entry addresses, output-section boundaries, load addresses, and segment permissions come from the script and its expressions. The linker can use `--emit-relocs` to retain relocation data and `--build-id`, `--gc-sections`, and `--eh-frame-hdr` when requested by a kernel build. The resulting files carry the layout that the boot loader and kernel expect.

24.3. Kernel addresses and calling conventions

`-mcmodel=kernel` encodes kernel symbol addresses as sign-extended 32-bit immediates, with `R_X86_64_32S` relocations. This is the addressing form for code linked into the upper two gigabytes of the virtual address space. The `-mno-sse` flag keeps the variadic argument prologue from saving XMM registers, while `-mgeneral-regs-only` rejects floating-point

expressions in kernel code. These flags affect instruction selection and function entry code; the ordinary System V argument classification remains the basis for separately compiled C objects.

25. GNU C and C11 extensions

Kernel and library sources are seldom written in plain C99. They rely on a group of GNU extensions and C11 additions, and SCCC implements the common ones within its single pass. Most of them reuse two small mechanisms: compiling an expression only for its type, and reading a group of tokens a second time.

25.1. Compiling for the type only

`sizeof expr` already compiled its operand and threw the code away. That step is now the pair `mark_code` and `drop_code`. A `Mark` records the length of the text buffer, the number of relocations, and the push count. `drop_code` restores all three and drops any label patch that points into the removed bytes. `typeof(expr)`, `_Alignof expr`, the controlling expression of `_Generic`, and the associations `_Generic` does not select are compiled this way. So are the second argument of `__builtin_expect` and the unchosen branch of `__builtin_choose_expr`.

```
#define max(a, b) ({ typeof(a) _a = (a); typeof(b) _b = (b); \
  _a > _b ? _a : _b; })
```

`_Generic` compares the type of its controlling expression with each association using `same_type`. Arrays and functions decay first, and type copies made by `aligned` are traced back through `origin`. The association that matches is compiled in place, and its value is the value of the whole expression. A `default` association may appear before the matching one, so its tokens are saved with `capture_expr`. When nothing else matches, `replay` pushes them back onto the source stack, ahead of the current token, and they are parsed again. The same pair of functions handles a range designator followed by a member designator, which is parsed once for each element of the range.

25.2. Statement expressions

`({ ... })` is recognized in `parse_primary` when a parenthesis is followed by a brace. The block is parsed statement by statement in a new scope. An expression statement whose semicolon is followed by the closing brace is the result: `rval` loads it into `RAX` before anything else is emitted. A label directly before the last statement is defined in place, so the value survives. The code and stack accounting are those of an ordinary block, so a statement expression may declare variables, contain loops, or `break` out of an enclosing loop.

25.3. Builtins

`__builtin_constant_p` never compiles its argument. Its tokens are captured, and the result is 1 when every identifier is an enumeration constant or a type name. Assignments, increments, the comma operator, and braces make the result 0, and the operand of `sizeof` is skipped. `__builtin_types_compatible_p` and `__builtin_offsetof` are computed by `builtin_const` from type records alone. All three, with `__builtin_choose_expr` and `__builtin_expect`, are also accepted by the constant-expression evaluator, so `offsetof` from `<stddef.h>` now works in array sizes, `case` labels, and static initializers.

25.4. Ranges

A case range `case lo ... hi:` stores both bounds. The dispatch sequence after the `switch` body subtracts `lo` from the controlling value and compares the difference with `hi - lo` as an unsigned number, so one `jbe` covers the whole range for signed and unsigned types alike:

```
switch (c) {
case '0' ... '9': return DIGIT;
case 'a' ... 'z': case 'A' ... 'Z': return LETTER;
}
```

A range designator `[lo ... hi] = v` initializes element `lo` and then copies it. For static data the bytes are copied in the data buffer, and every absolute relocation inside the first element is repeated for the others. For an automatic array one overlapping `rep movsb`, from element `lo` to element `lo + 1`, replicates the element across the range.

25.5. Labels as values

`&&label` emits `lea rax, [rip + disp]`. The displacement is patched like a jump, through the label's patch list, so it works before and after the label is defined. `goto *p` compiles `p` and emits `jmp rax`. A static array of label addresses is initialized with absolute relocations against the text section. Their addends are filled in when the label is defined, through a second list in `Label`. That is the table a threaded interpreter needs:

```
static const void *ops[] = { &&push, &&add, &&halt };
goto *ops[*pc++];
```

25.6. Attributes

`skip_attribute` reads each attribute in an `__attribute__((...))` list together with its argument tokens. The argument of `aligned` is a constant expression. `eval_tokens` evaluates it by pushing the tokens and a sentinel onto the source stack and calling `const_expr`, then restoring the current and peeked tokens, as `pp_eval` does for `#if`. String arguments of `section` and `alias` are interned. The results go into an `Attr` record that rides on the following token, next to the existing bit flags for `packed`, `interrupt`, and `weak`. `declspec` merges the records it passes over. Declarations merge in the one after the declarator, and `struct_decl` uses the ones after `struct`, after the tag, and after the closing brace.

Alignment is applied where storage is assigned. Structure layout takes the larger of a member's own alignment and its attribute, and the structure's attribute raises the alignment and so the rounded size. A typedef with `aligned` gets a copy of its type with the new alignment and `origin` pointing back. Globals record their alignment in `Sym`, and each section keeps the largest alignment placed in it, which `write_elf` uses when it assigns addresses. A function with `aligned(n)` is preceded by `int3` padding. Local variables are addressed from `rbp`, which is 16-byte aligned, so a local with a larger alignment is an error.

A weak symbol keeps the number of the file that declared it weak. When a second definition of a symbol arrives, a weak earlier definition gives way: the new body is compiled into the same `Sym`, and the earlier code remains in the file unused. A weak later definition is compiled into a discarded local symbol instead, which is the treatment library functions already received when the program defines them. An alias records the name of its target. At

the end of the file, `resolve_aliases` looks the name up among the file-scope symbols, and `write_elf` copies the target's section and offset before it resolves relocations.

25.7. Sections

A `section` attribute selects one of the `Sec` records in `Ctx`, created on first use with a kind of code, read-only, or data. A function is compiled into its section by swapping the section's buffer with `ctx->text` and setting `cur_text` for the duration of `parse_func_body`. A variable is defined by swapping with `ctx->data` and setting `cur_data`. Everything that emits code or data keeps working on `ctx->text` and `ctx->data` unchanged. Labels, jump patches, and the `sizeof` rollback all stay within the current buffer.

```
#define initcall(fn) static int (*const fn##_call)(void) \
  __attribute__((section("initcalls"), used)) = fn
extern int (*const __start_initcalls[])(void), (*const __stop_initcalls[])
(void);
```

Before relocations are resolved, `define_bounds` gives each undefined `__start_name` or `__stop_name` the address of the start or end of section `name`, as GNU ld does. Tables built in several files therefore need no registration code at run time. The `initcall` kernel boots this way.

26. Thread-local storage

`__thread` and `_Thread_local` declarations place objects in per-thread storage. The compiler separates initialized data from zero-filled storage into `.tdata` and `.tbss`. These are not normal globals at fixed process-wide addresses. The same identifier refers to a different copy when another thread uses it.

```
__thread int counter = 1;

int next_counter(void) {
    return ++counter;
}
```

SCCC uses the local-exec x86-64 TLS model. Loads and stores can address a TLS object relative to the FS segment, with an `R_X86_64_TPOFF32` relocation carrying its offset. For a variable's address the code uses the thread pointer and that same offset. The linker collects the TLS sections and describes their initialized bytes and total extent in a `PT_TLS` program header.

In a hosted executable, `__libc_init` reads the auxiliary vector and ELF TLS header, allocates a thread control block, copies initialized TLS values, clears the remaining bytes, and sets the FS base with `arch_prctl`. A freestanding kernel controls its own per-thread or per-CPU FS setup. This is also why TLS code in an independently compiled object can link correctly: the relocation is resolved against the linker's TLS layout rather than against an ordinary data address.

27. Self compilation and testing

The bootstrap rule builds two self-compiled generations and compares the bytes:

```
./sccc sccc.c -o sccc2
./sccc2 sccc.c -o sccc3
cmp sccc2 sccc3
```

For the source accompanying these notes the comparison succeeds. In this build the host-built compiler occupies 613400 bytes and the second and third generations each occupy 1184856 bytes. The latter two files are byte-identical. The first executable was built by GCC from the same C source, but the others were generated entirely by SCCC.

`make check` tests the language and the toolchain from several directions. The source tree contains 182 C-driver cases, 22 feature programs, and 47 example programs. The C driver compiles and runs small programs and checks diagnostics. The feature and example suites compile source directly and also through intermediate object files. `tests/link/` covers symbol resolution, weak and common definitions, TLS, and object linking. `tests/ldscript/` checks scripted section placement. The ABI fuzzer compiles calling-convention cases across SCCC and GCC. The assembler corpus checks instruction encodings, and the linker fuzzer exercises generated link scripts. Bare-metal tests boot the kernel examples in QEMU and Bochs.

The operating-system builds have their own targets:

```
make xv6          # run the xv6 user tests and compare with GCC
make xv6run       # boot xv6 to its interactive shell
make linuxrun     # compile and boot Linux with an SCCC-built /init
make busyboxrun   # build musl, BusyBox, and the Linux userspace
make busyboxcheck # compare SCCC and GCC userspace builds
```

The xv6 build uses the compiler for kernel C, assembly, object linking, and user programs. Its boot reaches `init: starting sh` and accepts commands at the `$` prompt. The automated xv6 script runs `usertests` and compares the output against a GCC-built reference. The Linux script builds `vmlinux` and boots it through QEMU's PVH entry. `make busyboxrun` builds Linux 5.4.302, musl 1.2.5, and BusyBox 1.36.1, then starts an interactive `ash` shell from the initramfs. The recorded session executes `ls`, `pwd`, and `reboot`. The library and shell differential tests compare their results with the GCC builds.

Byte identity demonstrates that the compiler reproduces its own output. The language, ABI, assembler, linker, bare-metal, and xv6 tests exercise different properties. Self-hosting checks a fixed point in the compiler build; compiling the operating system checks a larger body of C and assembly under a different runtime.

28. Source size and executable size

The compiler now contains 14977 lines in `sccc.c`; the C test driver contains 1326 lines. `builtin.h` contains 2373 lines of generated embedded headers and library text, together with the embedded boot sources used by the image writers. Those sources are compiled or assembled by SCCC when a build needs them.

The compiler source contains an AT&T syntax assembler, relocatable ELF object writer and reader, static-archive extraction, ABI argument classification, inline assembly, linker-script interpretation, TLS layout, ELF32 object links, and shared ELF construction. Separate compilation produces an object representation; mixed-source links require global symbol resolution; scripts place kernel sections at their virtual and physical load addresses. The

newest C front end also includes dynamic arrays, x87 `long double`, complex arithmetic, constant folding, dead-code elimination, and inline function expansion.

The self-compile timing on the author's x86-64 machine was approximately 0.08 seconds for the current `sccc.c`. TCC compiled the same source in 0.05 seconds, GCC at `-O0` in 2.0 seconds, and GCC at `-O2` in 13.8 seconds. These are compilation times for one source file on the same machine. They measure how quickly each compiler constructs the executable, not the running speed of the code it generates.

```
compiler       compile sccc.c
TCC                 0.05 s
SCCC                0.08 s
GCC -O0             2.0 s
GCC -O2            13.8 s
```

The host-compiled SCCC and self-compiled stages have different executable sizes because GCC and SCCC emit different instruction sequences. The bootstrap requires identity only between `sccc2` and `sccc3`. The new test commands measure assembler, linker, ABI, and operating-system behavior in addition to compiler reproduction.

29. Implementation notes

C expression instructions are emitted as the recursive-descent parser reads their operators. The lvalue state determines whether `RAX` contains a value or an address. Floating-point expression values are carried as bit patterns and transferred to SSE registers for arithmetic and for ABI calls. Type classification is used by both `parse_call` and `parse_func_body`, so separately compiled functions agree on the positions of their arguments and results.

The assembler is a second front end for the same instruction encoder. `assemble_unit` processes a source file; `assemble_snippet` is used by inline assembly. The code writer has to retain relocations when the output is an object, and it has to resolve them when the output is an executable. `write_object` implements the former path; `load_object` reads external ELF objects. Archive extraction determines which of those objects participate in a link.

The script parser and layout engine appear in the `ld_` functions. The parser records assignments, input-section patterns, and output-section rules. The matching and layout passes assign file offsets, virtual addresses, and load addresses; `ld_write_elf` writes the resulting ELF file. `write_elf` remains the driver for the original output formats and for links without a script. Image construction uses the assembly sources under `lib/boot` and the same encoder.

The xv6 build traverses C compilation, preprocessed assembly, separate object emission, archive and symbol handling, the System V ABI, scripted kernel layout, and executable writing. The bare-metal kernels exercise early boot and devices; xv6 adds processes, a file system, and separately linked user programs. The Linux build adds Kbuild option parsing, relocatable intermediate links, ELF32 entry code, kernel address models, and shared vDSO layout. Building musl and BusyBox adds an independently developed C runtime, a shell, system utilities, and interactive userspace on top of the SCCC-built kernel.

30. Conclusion

SCCC is a self-compiling x86-64 C toolchain. Its C parser emits machine instructions directly; its assembler reads `.s`, `.S`, and inline assembly; its object writer produces ELF relocatable files; and its linker reads objects and archives, processes linker scripts, resolves

relocations, and constructs final executables. It also writes ELF32 boot objects and shared ELF images, handles kernel address models, and builds a Linux vDSO. Its functions use the System V calling convention.

The compiler compiles itself to identical second- and third-stage executables. It builds xv6-x86_64 and boots it to a shell. It also builds Linux 5.4.302, musl 1.2.5, and BusyBox 1.36.1. The Linux system initializes its devices, starts `/init`, enters an interactive `ash` shell, runs commands from the initramfs, and reboots. SCCC provides the compiler, assembler, linker, and runtime construction needed to take these sources from C and assembly to running operating systems.

Availability. The source code of the compiler, library, hosted and bare-metal tests, kernel examples, and bootstrap Makefile carry the GPL-3.0-only license and are distributed at: `https://github.com/gaidardzhiev/sccc`